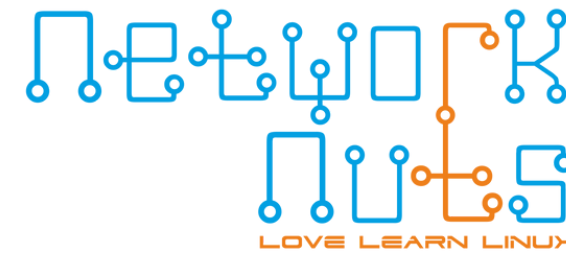


Kubernetes & Helm

MANAGING KUBERNETES WITH HELM

Need of Helm



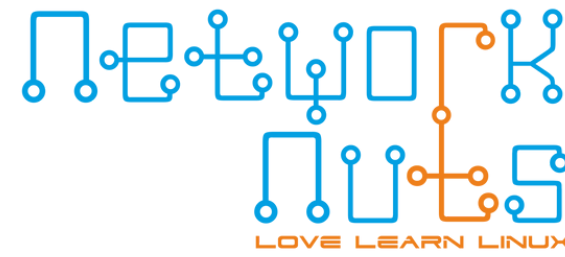
Understanding kubernetes & helm

Teams face tremendous pressure to ensure that applications are lightweight and scalable. Applications must also be highly available and able to withstand varying loads. Industry has started using tools such as Kubernetes to orchestrate and scale their containerized microservices.

Kubernetes, has its own set of challenges. While it is an effective container orchestration tool, it presents a steep learning curve that can be difficult for teams to overcome. One tool that helps simplify the challenges of running workloads on Kubernetes is Helm.

Helm allows users to more simply deploy and manage the life cycle of Kubernetes applications. It abstracts many of the complexities behind configuring Kubernetes applications and allows teams to be more productive on the platform.

Need of Helm



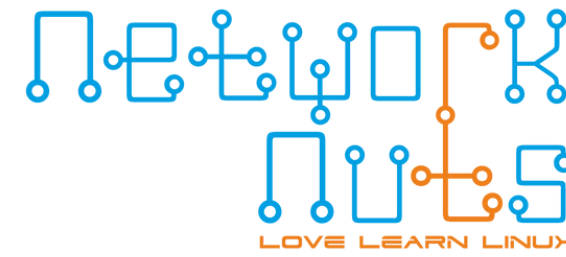
Deploying Kubernetes Application

Deploying an application on Kubernetes is similar to deploying an application outside of Kubernetes. All applications, whether containerized or not, must consider the following configuration details:

- Networking
- Persistent storage and file mounts
- Resource allocation
- Availability and redundancy
- Runtime configuration
- Security

Configuring these details on Kubernetes is done by interacting with the Kubernetes application programming interface (API). The Kubernetes API serves as a set of endpoints that can be interacted with to view, modify, or delete different Kubernetes resources, many of which are used to configure different details of an application.

Need of Helm

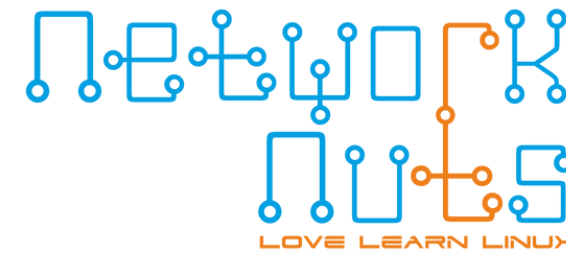


Approaches to Resource Management

To deploy an application on Kubernetes, we need to interact with the Kubernetes API to create resources. **kubectl** is the tool we use to talk to the Kubernetes API.

- kubectl create
- kubectl apply
- kubectl describe
- kubectl edit
- kubectl delete

Need of Helm



Challenges in Resource Configurations

Types of Kubernetes Resources:

There are many different types of resources in Kubernetes. We need to be able to determine which resources are required to deploy applications, and need to understand them at a deep enough level to configure them appropriately. This requires a lot of knowledge of and training on the platform.

Keeping Live & Local State in Sync:

The configuration defined in source control, referred to as the **local state**, is then created by applying them to the Kubernetes environment, and the resources become live or enter what can be called a **live state** (live object configuration). But what happens when developers need to make changes to their resources? The right way is to modify the files in source control and apply the changes to synchronize the local state to the live state. However, this isn't what always ends up happening. It is often simpler, in the short term, to modify the live resource in place with `kubectl edit` or `kubectl patch` and completely skip over modifying the local files. This results in state inconsistency between local and live states and is an act that makes scaling on Kubernetes difficult.

Need of Helm

Challenges in Resource Configurations

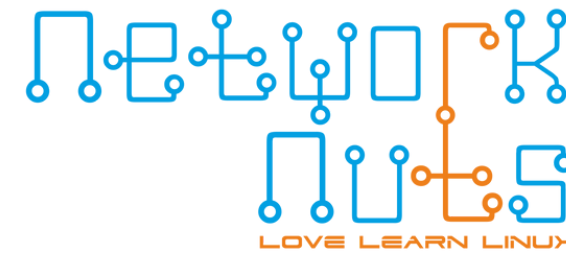
Hard to Manage Application Lifecycle:

The initial installation would create what we refer to here as version 1 of an application. An upgrade, then, can be thought of as a modification to one or many of those Kubernetes resources. Which is version 2 now. Next update in the application will bump the version count to version 3. As newer versions of an application continue to be rolled out onto Kubernetes, it becomes more difficult to keep track of changes that have occurred across relevant API resources. Kubernetes, in most cases, does not have an inherent way of keeping a history of changes. While this makes upgrades harder to keep track of, it also makes restoring a prior version of an application much more difficult.

Resource Files are Static:

Kubernetes resource files are not natively designed to be parameterized. Resource files are largely designed to be written out in full before being applied, and the contents remain the **source of truth (SOT)** until the file is modified. Static files lend themselves to becoming boilerplate. **Boilerplate** represents text or code that remains largely consistent in different but similar contexts. This becomes an issue if developers manage multiple different applications, where they could potentially manage multiple different deployment resources, multiple different services, and so on. In comparing the different applications' resource files, you may find large numbers of similar YAML configurations between them.

Need of Helm



Challenges in Resource Configurations

```
apiVersion: v1
kind: Deployment
metadata:
  name: my-k8s-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-k8s-app
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  template:
    metadata:
      labels:
        app: my-k8s-app
    spec:
      containers:
        - image: my-k8s-app:v1
          imagePullPolicy: IfNotPresent
          name: app
```

```
apiVersion: v1
kind: Deployment
metadata:
  name: your-k8s-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: your-k8s-app
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  template:
    metadata:
      labels:
        app: your-k8s-app
    spec:
      containers:
        - image: your-k8s-app:v3
          imagePullPolicy: IfNotPresent
          name: app
```

The blue text denotes lines that are boilerplate, while the red text denotes lines that are unique. Both files are almost exactly the same. When managing files that are as similar as this, boilerplate becomes a major headache for teams managing their applications in a declarative fashion.

Welcome Helm

Helm to the rescue

Kubernetes community discovered that creating and maintaining Kubernetes resources to deploy applications is difficult. This prompted the development of a simple yet powerful tool that would allow teams to overcome the challenges posed by deploying applications on Kubernetes. The tool that was created is called Helm. **Helm** is an open source tool used for packaging and deploying applications on Kubernetes. It is often referred to as the Kubernetes package manager because of its similarities to any other package manager you would find on your favorite operating system (OS). Helm is widely used throughout the Kubernetes community and is a CNCF graduated project.

What are Package Managers?

Package managers are used to simplify the process of installing, upgrading, reverting, and removing a system's applications. These applications are defined as packages that contain metadata around target software and its dependencies.

The idea behind package managers is simple. First, the user passes the name of a software package as an argument. The package manager then performs a lookup against a repository to see whether that package exists. If it is found, the package manager installs the application defined by the package and its dependencies to specified locations on the system.

Like: `yum install httpd` OR `apt install vim`

Welcome Helm

The Kubernetes Package Manager

Helm was designed to provide an experience similar to that of package managers. **dnf** operates on RPM Package Manager (RPM) packages that provide executables, dependency information, and metadata. Helm, on the other hand, works with charts. A **Helm chart** can be thought of as a Kubernetes package. Charts contain the declarative Kubernetes resource files required to deploy an application. Similar to an RPM package, it can also declare one or more dependencies that the application needs in order to run. Helm then relies on repositories to provide widespread access to charts. Chart developers create declarative YAML files, package them into charts, and publish them to chart repositories. End users then use Helm to search for existing charts to deploy onto Kubernetes, similar to how end users of dnf / apt will search for RPM / DEB packages to deploy to Fedora / Ubuntu.

Welcome Helm

The Kubernetes Package Manager

Some Examples:

```
# helm install redis bitnami/redis --namespace=my-redis
```

This will install the redis chart from the bitnami repository to a Kubernetes namespace called my-redis. This installation would be referred to as the initial revision, or the initial installation of a Helm chart.

```
# helm upgrade redis bitnami/redis --namespace=my-redis
```

This would upgrade redis to meet the specification defined by the newer redis chart.

```
# helm rollback redis 1 --namespace=my-redis
```

This command would roll redis back to its first revision.

```
# helm uninstall redis --namespace=my-redis
```

This will remove redis altogether with the uninstall command,

Welcome Helm

The Kubernetes Package Manager

dnf subcommands	Helm subcommands	Purpose
<code>install</code>	<code>Install</code>	Install an application and its dependencies.
<code>upgrade</code>	<code>Upgrade</code>	Upgrade an application to a newer version. Upgrade dependencies as specified by the target package.
<code>downgrade</code>	<code>rollback</code>	Revert an application to a previous version. Revert dependencies as specified by the target package.
<code>remove</code>	<code>uninstall</code>	Delete an application. Each tool has a different philosophy around handling dependencies.

Welcome Helm

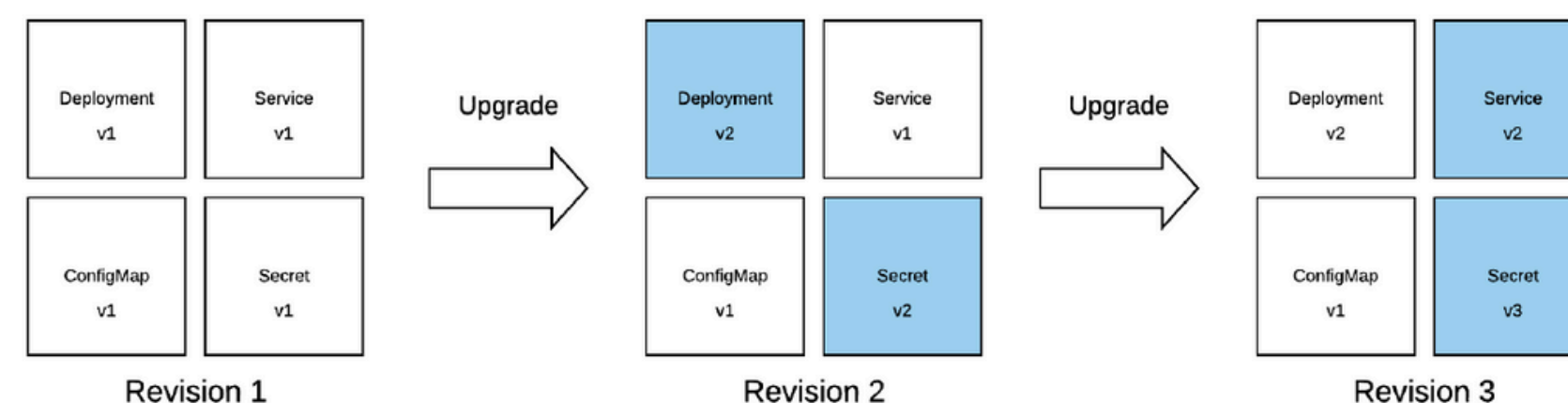
Benefits

Removing the complexity of Kubernetes resources:

No need to write YAML for resources. Simply, search for WordPress charts from upstream chart repositories. These charts have already been written by chart developers in the community and would already contain the declarative configuration required to deploy WordPress and a backing database. Vendor-owned chart repositories also tend to be well maintained, so teams using charts from them would not need to worry about keeping Kubernetes resources up to date. So developer just need to find the chart and use it, in a similar way to any other package manager.

Maintaining History of Revisions:

Helm has a concept called release history. When a Helm chart is installed for the first time, Helm adds that initial revision to the history. The history is further modified as revisions increase via upgrades, keeping various snapshots of how the application was configured at varying revisions.



Welcome Helm

Benefits

Declarative & Dynamic Resources:

Kubernetes resources are static and cannot be parameterized. Helm remove these issues by introducing **values** and **templates**. Values can be thought of as parameters for charts. Templates are dynamically generated files based on a given set of values. These two constructs give chart developers the ability to write Kubernetes resources that are generated based on values that end users provide.

Local and live state synchronization:

Using Helm's **values** construct, users can supply configuration changes across an application's life cycle by managing a small number of parameters instead of multiple full-length YAML resources. When the local state (values/parameters) is updated, Helm propagates the configuration change out to the relevant resources in Kubernetes. This workflow keeps Helm in control of managing intricate Kubernetes details and encourages users to manage the state locally instead of updating live resources directly.

Welcome Helm

Benefits

Intelligent Way of Deploying Resources:

Helm simplifies application deployments by having a pre-determined order in which Kubernetes resources need to be created. This ordering exists to ensure that dependent resources are deployed first. For example, Secret instances and ConfigMap instances should be created before deployments, since a deployment would likely consume those resources as volumes. Helm performs this ordering without any interaction from the user, so this complexity is abstracted and prevents users from needing to understand the order in which resources should be applied.

Automated Life-cycle Hooks:

Helm provides the ability to define life cycle hooks. Life cycle hooks are actions that take place automatically at different stages of an application's life cycle. They can be used to do things such as the following - perform data backup on upgrade - restore data on rollback - validate before installing.